Methodology and Architectural Justification

RL-Based Adaptive Modulation and Coding (v6, OLLA-Relative Encoding)

Supporting Technical Documentation

July 2026

0.1 RL-Based Adaptive Modulation and Coding (v6, OLLA-Relative Encoding)

Version Evolution, Design Audit, and Architectural Rationale (v1 → v6)

Case 1 — Reinforcement Learning for Link Adaptation in LTE/5G-NR Prepared as supporting technical documentation for conference submission

—

Abstract

This document is the methodology and architectural-justification companion to the v6 ("OLLA-relative") reinforcement-learning link-adaptation stack developed for this project. Its purpose is narrower than a full paper: rather than re-deriving the system model from scratch, it traces the six-version evolution of the codebase (v1 through v6), states precisely which conceptual or architectural defect motivated each revision, shows the fix in the code, and then gives a complete methodological account of v6 — the version used for the camera-ready results. The central architectural claim of v6 is that every learner (tabular Q-learning, tabular actor-critic, and a shared-parameter deep A2C) is re-parameterised to act relative to a per-user Outer-Loop Link Adaptation (OLLA) controller that is updated on realised ACK/NACK feedback, identical to the one used by the OLLA baseline itself. Action zero is defined to be exactly the OLLA decision, so every learner starts at OLLA performance and can only improve on it by learning state-conditional corrections that a scalar reactive control loop cannot represent. This removes the last informational asymmetry left over after the v5 audit (in which OLLA had access to ACK/NACK feedback the RL agents did not) and makes "RL beats OLLA" a claim that, if it holds, cannot be dismissed as a comparator handicap. Averaged over five training seeds and thirty held-out channel traces, the resulting Deep A2C policy beats OLLA in cell throughput at every tested CQI-feedback delay (0, 2, 4, 6, and 8 scheduling intervals) under the primary proportional-fair scheduler; after Holm–Bonferroni correction across the five-delay family the margin over OLLA is statistically significant (Wilcoxon signed-rank, paired across traces) at two of the five delays — delay 2 and delay 6 — with three delays nominally significant before correction. Against the fixed-mapping baseline the Deep A2C margin is significant at every delay and survives correction trivially.

—

1 1. Introduction and Purpose of This Document

The project implements reinforcement-learning-based Adaptive Modulation and Coding (AMC) for a multi-user LTE/5G-NR downlink and compares it against two standard link-adaptation baselines: a fixed-mapping ("genie-observed-SINR") baseline and an industry-standard Outer-Loop Link Adaptation (OLLA) controller. The codebase went through six labelled versions (v1–v6), each triggered by a specific conceptual or architectural problem discovered in the previous one — not by routine refactoring. Several of these problems were severe enough that they invalidated the headline result of their version outright (a broken argmax tie-break that made three algorithms collapse; a scheduler that leaked ground-truth channel state to itself; a reward function that rewarded every user for lying about its channel quality). This document exists so that a reader of the conference paper — or a future maintainer of the code — can see not just what v6 does, but why every structural choice in it is a direct, traceable response to a documented failure mode, rather than an arbitrary design decision.

Section 2 gives a one-page map of the six versions. Section 3 is the detailed audit: for each version, what was conceptually or architecturally wrong, how it was diagnosed, and what changed. Section 4 gives the complete methodology of v6 in isolation — system model, MDP formulation, algorithms, training procedure — as it would appear in a paper's methods section. Section 5 is the architectural justification proper: why the OLLA-relative design is the correct fix for the specific asymmetry left over after v5, argued from first principles, not just from

the fact that it produced better numbers. Section 6 reports the empirical validation actually obtained. Section 7 states remaining threats to validity. Section 8 gives the reproducibility artefacts (hyperparameter tables, file map, run commands).

—

2 2. Evolution Overview: v1 → v6

Table 1 summarises the six versions. Each row's "headline evidence" is the number that exposed the problem fixed in the next row — the story of this project is best told through its own broken results, since each one pinpoints exactly what was wrong.

—

3 3. Version-by-Version Conceptual and Architectural Audit

This section is organised by the same eight conceptual/architectural fault categories (C1–C8) identified during the internal design audit of the pre-v4 codebase, plus the three additional faults found during the v5 audit and the one remaining asymmetry resolved by v6. Faults are attributed to the version in which they were introduced or first diagnosed.

3.1 3.1 v1 — Original per-user tables/networks

v1 is the direct implementation of the textbook single-agent RL-AMC formulation (state = (CQI, ΔCQI, SINR), action = signed CQI offset, reward = throughput) applied independently to each of the case studies (Case 1: single user, radial mobility; Case 2: 5 users, PF scheduling; Case 3: 5 users, round robin). Two architectural defects dominate its failure on the multi-user cases.

- **Zero-initialised argmax tie-break.** With all Q-values or preferences initialised at zero and a scheduler (PF) that gives roughly 80% of transitions zero reward, most state–action cells are never updated. MATLAB's `max()` returns the first maximising index on a tie, which corresponds to the most negative offset action — so greedy action-selection in unvisited states silently drives every user's CQI to its floor. This is indistinguishable from a converged "conservative" policy unless the tie-breaking rule is inspected directly.
- **Independent per-user learning** fragments the experience five ways in the multi-user cases: each user's table sees only 1/N of the transitions a shared table would see, and none of the tables can transfer what one user's trajectory teaches about another's.
- **Reward decoupling in Case 1** (the single-user case): the transport block size (TBS) credited to the agent was read from the baseline's SINR→MCS mapping while the block-error rate charged to the agent came from the agent's own MCS choice. The optimal policy under this bug is degenerate — drive the reported CQI to its minimum so BLER → 0 while still collecting the baseline's transport block — and any apparent "gain over baseline" under this bug is an artefact of the decoupling, not of link adaptation.

Headline evidence: cell throughput under PF was Baseline 9.108, Q-Learning 3.932, Tabular AC 2.550, Deep A2C 5.307 Mbps — all three RL schemes below the baseline, with A2C "least bad." Training diagnostics showed the A2C learning curve oscillating without an upward trend and its gradient clipped 14,756 times over 200 episodes, indicating a saturated optimiser rather than a converging one.

3.2 3.2 v2 — Bias-initialisation and reward shaping (case_2_pf)

v2 attempted to fix the v1 collapse by biasing the tabular initial values toward the no-op (offset-zero) action and adding reward shaping intended to make agents "converge to the baseline first, then improve." This rests on a false premise that is the seed of the v3 regression described next: the action space is differential (an offset from the current CQI), so "choose the no-op action" does not mean "behave like the baseline" — it means "freeze the CQI at whatever value the episode started with." A no-op-biased policy therefore stops tracking the SINR as a mobile user moves, which is precisely the behaviour link adaptation exists to prevent. v2 partially recovered throughput relative to v1 but planted the freeze bug that v3 amplifies.

3.3 3.3 v3 — Deeper bias-init and explicit imitation bonus

v3 pushed the v2 idea further: stronger bias-initialisation (actor bias vector $[-1, -0.5, +1.5, -0.5, -1]$, Q-table init $+0.5$ on the no-op cell) plus an explicit scalar bonus paid every time an agent selects the no-op action, applied identically to Q-Learning, Tabular AC, and Deep A2C. Because the freeze-bug premise from §3.2 was never corrected, the shaping succeeded exactly as designed — and that is the problem: the more strongly an algorithm was biased toward the no-op action, the more often it chose to freeze its CQI regardless of channel state, and the worse it performed. Tabular AC received both the strongest bias and the explicit bonus and became the worst performer of

Table 1: One-line summary of each version, the motivating evidence, and the verdict.

Ver.	Core structural idea	Headline evidence	Verdict
v1	Per-user Q-tables / per-user actor-critic / per-user networks; action = absolute CQI choice.	Multi-user: all three RL schemes far below baseline (A2C 5.31 vs Baseline 9.11 Mbps, PF).	Broken — zero-init argmax bias; no per-user table sees enough data.
v2	Bias-initialise tables toward the no-op action; reward shaping to encourage "imitate baseline" behaviour.	Partial recovery, but a new failure mode appears: no-op is silently reinterpreted as "freeze current CQI" rather than "behave like baseline."	Partially fixed one problem, introduced another (freeze bug).
v3	Deeper bias-init + explicit imitation bonus for the no-op action, applied to all three learners.	Worst-performing version: Tab-AC (strongest bias + bonus) drops to 2.55 Mbps, below its own v1 number.	Regression — the shaping intended to help was the direct cause of the collapse.
v4	Baseline-relative action encoding (action = offset from the fixed-mapping CQI); shared network/table across all N users (CTDE); full audit-driven rewrite of channel, BLER, reward, and A2C training internals.	First version to beat the fixed-mapping baseline: Deep A2C 12.32 vs Baseline 12.21 Mbps (initial run); OLLA baseline added for reference.	First honest positive result; motivates a full audit-driven rebuild (v5).
v5	v4 architecture + three-stage audit fix: (i) proportional-fair (PF) scheduler stops seeing true BLER, (ii) cooperative cell-level reward replaces per-user reward, (iii) CQI thresholds re-anchored to calibrated BLER=0.1 points.	Post-audit, honest result ($N = 10$, PF, delay 4): OLLA +30.5%, Deep A2C +26.4% (not significant vs. OLLA), Tab-AC +16.2%, QL -2%.	Rigorous and honest, but RL only matches OLLA — does not beat it.
v6	OLLA-relative action encoding: every learner carries the identical per-user OLLA loop as the comparator; action $0 \equiv$ OLLA; OLLA offset is a state feature.	Deep A2C beats OLLA in cell throughput at every tested CQI delay $\{0, 2, 4, 6, 8\}$; significant at 3 of 5 delays (Wilcoxon, paired).	Current best — information-symmetric comparison; genuine, defensible RL gain.

the three (2.55 Mbps, below even its own v1 result); Deep A2C's warm-up blending of the biased policy with random exploration preserved some SINR-tracking behaviour and made it the best of a bad set. The conceptual lesson generalises beyond this codebase: reward shaping and initialisation bias are only safe when the shaped behaviour is verified against the actual semantics of the action space, not against an intuitive label for the action.

3.4 3.4 v4 — Baseline-relative encoding and the eight-point structural rebuild

v4 is the version that first beat the fixed-mapping baseline, and it did so by replacing the failed heuristics of v2/v3 with a structural fix plus a comprehensive rebuild driven by an eight-point internal design audit (categories C1–C8 below). The single most important architectural change is baseline-relative action encoding: the action becomes an offset from the CQI a fixed mapping would select on the observed SINR, so offset 0 is defined to equal the baseline exactly by construction — not by learned imitation. A learner therefore starts at baseline performance for free and only has to learn a correction, which removes the entire freeze-bug failure mode without needing bias initialisation or imitation bonuses at all (both were deleted).

The remaining seven fixes bundled into v4, each addressing one audit category:

- **C1 — Reward decoupling (Case 1).** Both TBS and BLER are now computed from the agent's own selected MCS in every case, eliminating the degenerate

"drive CQI to the floor" optimum.

- **C2 — Genie baseline / no headroom to exploit.** The pre-v4 baseline observed the true, instantaneous, unquantised SINR every TTI, so RL could at best imitate it and had no informational edge to exploit. v4 introduces CQI feedback delay, quantisation, and measurement noise ("imperfect CQI"), which is also the point at which an OLLA baseline is added as a second, more realistic comparator.

- **C3 — No-op imitation fallacy.** Bias initialisation and imitation bonuses removed; policies start uniform / zero-initialised (see §3.2–3.3).

- **C4 — Divergent per-algorithm objectives.** Reward, discount factor, and shaping terms are unified into one MDP definition shared by all three learners; only optimiser hyperparameters (learning rate, entropy coefficient) are allowed to differ per algorithm.

- **C5 — Miscalibrated BLER model.** Per-MCS BLER steepness (previously dead code, overridden by a flat 0.2 dB^{-1} slope) is restored and the curves re-anchored so BLER = 0.1 at each CQI's design threshold, matching the LTE/NR 10% BLER target, instead of BLER $\approx$ 0.5 with 10–20 dB wide transition regions.

- **C6/C7 — Channel and timescale realism.** Path loss, shadowing, and fading are replaced with a 3GPP TR 38.901 UMa-NLOS model (correct exponents, log-normal Gudmundson-correlated shadowing at σ = 6 dB, AR(1) block fading), thermal-noise-plus-noise-figure receiver noise, and an explicit interference margin so the quantity used is defensibly an SINR rather than an SNR; all ad hoc clipping of fading/shadowing values is removed.

- **A2C training pathology.** The v1–v3 A2C trainer suffered from unnormalised reward magnitudes combined with per-tensor gradient clipping at a fixed norm of 1.0, which saturates Adam (14,756 clips logged in a 200-episode run) and produces a flat, non-improving learning curve. v4 introduces reward normalisation, per-batch advantage standardisation, a single global gradient-norm clip, time-limit-aware value bootstrapping (Pardo et al. 2018) instead of bootstrapping zero at the artificial episode horizon, and an entropy coefficient annealed from 0.05 to 0.001 so the stochastic training policy converges toward the deterministic policy used at evaluation time.

v4's first full run (before an internal v4.1 refinement) produced Baseline 12.21, OLLA 12.08, Q-Learning 7.00, Tabular AC 11.76, Deep A2C 12.32 Mbps — A2C beat OLLA at p = 0.005 but was statistically tied with the baseline (p = 0.117), and Q-Learning had collapsed with a CQI tracking error of 3.79. Diagnosis attributed this to three further issues, fixed within v4.1: at a 2-interval feedback delay the baseline's own CQI error was dominated by unbiased (hence non-compensable) measurement noise, so the delay was raised to 4 intervals where the correlated, compensable component of the error dominates; the state carried ΔCQI relative to the agent's own past actions rather than an observable SINR trend, so channel drift was invisible to the policy and an explicit observed-SINR-trend feature was added; and Q-Learning's collapse was traced to the same zero-init argmax tie-break bug from v1, reintroduced because the fix had not been carried over — corrected with a permanent no-op-preferring tie-break (`argmaxNoOp`) used in both training and evaluation from v4 onward.

3.5 3.5 v5 — The three-stage post-v4 audit

v5 kept the v4 architecture (baseline-relative encoding, shared parameters, imperfect CQI, unified MDP, fixed A2C training) and layered a rigorous multi-seed, held-out-trace statistical evaluation protocol on top of it (95% confidence intervals, paired Wilcoxon signed-rank tests, Jain fairness index, edge-user throughput, CQI tracking error). That evaluation protocol is precisely what surfaced three further defects that would not have been visible from a single point estimate.

3.5.1 3.5.1 The PF scheduler was a genie (`pf_metric`)

The proportional-fair scheduler's scheduling potential was computed as (1 − BLER_true) × TBS, i.e. from the true, instantaneous block-error probability of the transmission that had not happened yet. A real MAC-layer scheduler cannot know this; it can only know the transport block size implied by the reported CQI. Because the scheduler could see which TTIs would fail, it systematically avoided scheduling exactly those TTIs — which inflated PF-scheduled throughput for every policy, disproportionately shielded aggressive policies from their own errors (their failures simply never got scheduled), and let any outer-loop offset ratchet upward without correction. The bug was caught because a warm-started OLLA baseline reached +164% over the fixed-mapping baseline — a magnitude that is physically impossible for a reactive link-adaptation loop and could only be produced if the scheduler itself possessed oracle information. The fix (`pfPotential_v5.m`, `env.pf_metric = 'reported'`) computes scheduling potential from the reported CQI's implied TBS only; the pre-fix behaviour is retained as an ex-

plicit legacy mode (`env.pf_metric = 'oracle'`) purely so that the invalid pre-audit numbers remain reproducible for comparison, never for the paper's reported results.

3.5.2 3.5.2 Per-user reward created a CQI-inflation prisoner's dilemma (`reward_agg`)

Once the PF scheduler was made honest, a second pathology appeared under per-user ("selfish") reward: because PF schedules the user whose reported CQI implies the best rate relative to its recent average, every agent has an individual incentive to over-report its channel quality to win scheduling slots, even though doing so raises its own block-error rate. When all agents learn this simultaneously, the system converges to a mutual-over-reporting equilibrium that collapses cell throughput (observed as a −31% regression relative to the baseline under honest PF). This is a textbook multi-agent social dilemma, not a bug in any single agent's learning rule. The fix, `env.reward_agg = 'cell'`, gives every user the identical per-TTI reward equal to total cell goodput — a cooperative, centralised-training/decentralised-execution (CTDE) reward that removes the incentive to inflate, since inflating one's own report can no longer increase one's own reward independently of its effect on the whole cell.

3.5.3 3.5.3 CQI-threshold miscalibration (2 dB) against the BLER model

The CQI-to-SINR threshold table used by the fixed mapping and by the reference-CQI computation had not been re-derived after the v4 BLER re-calibration (§3.4, C5); several mid-range CQI thresholds were up to 2.3 dB more aggressive than the SINR at which the BLER model actually crosses the 10% design point (e.g. CQI 11 switched at 11.7 dB but its own calibrated 10%-BLER point was 14.0 dB), which drove the baseline's realised BLER toward 0.45 rather than the intended 0.10. The thresholds were re-anchored so that every CQI's switching point coincides with its own BLER = 0.1 point, closing a gap between two model components that had previously been calibrated independently.

With all three fixes applied, the honest, statistically supported v5 result at N = 10 users, PF scheduling, and a 4-interval feedback delay was: OLLA +30.5% over baseline, Deep A2C +26.4% (not statistically significant against OLLA), Tabular AC +16.2%, Q-Learning −2%. This is the result that motivates v6: after three rounds of audit-driven fixes, the best RL policy matched but did not beat the industry-standard OLLA controller.

3.6 3.6 v6 — The residual asymmetry and the OLLA-relative fix

The v5 result left one structural asymmetry unaddressed, and it is informational rather than statistical: OLLA's outer loop is updated every scheduled TTI from realised ACK/NACK feedback on the actual transmission, whereas the RL agents were confined to offsets of ±2 CQI steps around a fixed, static SINR-to-CQI mapping and never saw that ACK/NACK stream at all. OLLA is, in effect, allowed to use information the RL agents structurally cannot access, so a null result ("RL just matches OLLA") is uninformative — it could equally mean RL has learned everything useful and the informational asymmetry accounts for the rest of the gap, or it could mean RL has learned nothing beyond noise. v6 removes the asymmetry directly rather than compensating for it statistically:

- Every learner (Q-Learning, Tabular Actor-Critic, Deep A2C) carries the identical per-user OLLA outer loop as the OLLA comparator itself — the offset is updated by the same rule, from the same realised ACK/NACK feedback, only on TTIs where that user is actually scheduled.
- The reference CQI seen and acted upon by every learner is `baseCQI(observed_SINR + olla_offset)`; the learned action is defined as an offset from that OLLA-corrected reference, so action 0 is now defined to be exactly the OLLA decision, not an approximation of it.
- The OLLA offset itself is exposed as a state feature (continuous for the deep agent; five bins for the tabular agents), so every learner can condition its correction on the outer-loop state it is acting around.

This is a superset-of-OLLA construction: if learning fails entirely, the tie-break/entropy-annealing behaviour of each trainer collapses toward the no-op action, i.e. toward OLLA itself, so v6 cannot perform structurally worse than OLLA by design. If learning succeeds, any measured margin over OLLA is a genuine learned improvement — specifically, a state-conditional (trend-aware) correction that a scalar reactive control loop cannot represent — and the comparison can no longer be characterised as a strawman in either direction. Sections 4 and 5 give the full mechanics and the argument for why this is the correct fix, not merely a fix that happened to produce better numbers.

—

4 4. v6 Methodology

This section states the v6 system model and MDP formulation completely and self-containedly, in the form it

would take in a paper's methods section. Everything in this section not explicitly marked "v6 change" is inherited unmodified from the audited v5 stack (see §3.4–3.5) and must not be reverted.

4.1 4.1 System model

The scenario is a single cell with N users (N = 10 for all headline results, under the full 5-seed × 30-trace camera-ready protocol; N ∈ 5, 20, 50 are **preliminary** scaling studies run at a reduced 3-seed budget and treated throughout as secondary, lower-confidence sensitivity checks — not confirmed results) under random-waypoint mobility at 5 m/s within a 1000 m square cell, base station at the centre, carrier frequency 2 GHz, base-station height 25 m, UE height 1.5 m. The channel follows the 3GPP TR 38.901 urban-macro non-line-of-sight (UMa-NLOS) path-loss model with log-normal shadow fading (σ = 6 dB, Gudmundson spatial correlation distance 50 m) and AR(1)-correlated block fading (correlation coefficient 0.95 per scheduling interval). Transmit power is 43 dBm with 8 dBi antenna gain, an explicit 6 dB interference margin (making the reported quantity a genuine SINR rather than an SNR), 20 dB outdoor-to-indoor penetration loss, 9 dB receiver noise figure, over a 10 MHz / 110 resource-block channel.

CQI feedback is imperfect: the base station observes SINR delayed by 4 scheduling intervals and corrupted by 1.0 dB (σ) measurement noise, then quantised to a 15-level CQI via a threshold table anchored to the BLER = 0.1 point of each corresponding MCS (§3.5.3). Three scheduling disciplines are supported — proportional fair (PF, primary), round robin (RR), and equal-share (EQ) — and training samples from a weighted mix of all three (50% PF / 25% RR / 25% EQ) so that policies generalise across schedulers rather than overfitting to one; evaluation reports each scheduler separately plus a robustness ratio across them.

4.2 4.2 OLLA outer loop (shared by comparator and every learner) — v6

Every user, regardless of which policy controls its offset, carries an independent OLLA state variable $\Delta(u)$ updated once per scheduled TTI:

- $\Delta(u) \leftarrow \Delta(u) + \delta_\text{up}$ if the transmission at the sent CQI is acknowledged (simulated as `rand > BLER_true`)
- $\Delta(u) \leftarrow \Delta(u) - \delta_\text{down}$ otherwise (a simulated NACK)

with δ_down = 0.5 dB and $\delta_\text{up} = \delta_\text{down} \cdot \text{target}/(1 - \text{target})$, target = 0.10, which is the standard step-size ratio that makes the loop's stationary point track the 10% BLER design target. The OLLA-corrected reference CQI for user u at time t is `ref(u,t) = baseCQI(observed_SINR(u,t) + Δ(u,t))`, computed identically for the OLLA baseline and for every RL learner.

4.3 4.3 Markov Decision Process

4.3.1 4.3.1 State

The state observed by every learner at time t for user u is the tuple `(ref_CQI, d_ref, obs_SINR, obs_SINR_trend, olla_offset, scheduler_regime)`:

- **ref_CQI** — the OLLA-corrected reference CQI defined in §4.2 (range 1–15; 15 tabular bins).
- **d_ref** — the change in reference CQI since the previous interval, clipped to [−5, +5] (11 tabular bins).
- **obs_SINR** — the delayed, noisy observed SINR (20 bins spanning −10 to +30 dB for tabular learners; a continuous, normalised input for the deep learner).
- **obs_SINR_trend** — the first difference of the observed SINR between consecutive intervals (an observable quantity, unlike ΔCQI computed from the agent's own past actions); 3 bins for tabular learners, a continuous input for the deep learner.
- **olla_offset** — the same $\Delta(u)$ from §4.2, discretised into 5 bins at edges −3, −1, +1, +3 dB for tabular learners, or included as a continuous, normalised feature for the deep learner. *This feature is new in v6.*
- **scheduler_regime** — a one-hot encoding of which scheduler (PF/RR/EQ) is currently active, observable by the agent (domain-randomisation-with-observable-context, inherited from v5.2).

The tabular state space is therefore 15 × 11 × 20 × 3 × 5 × 3 (scheduler contexts) ≈ 148,500 cells per action; the deep-network state is a 5+|schedulers| = 8-dimensional real-valued feature vector.

4.3.2 4.3.2 Action

The action set is A = −2, −1, 0, +1, +2, interpreted as an offset in CQI steps from the OLLA-corrected reference: `sent_CQI = clip(ref_CQI + a, 1, 15)`. Action 0 is, by construction, exactly the OLLA decision — this is the defining structural property of v6 relative to v5, where action 0 was an offset from a static fixed mapping rather than from a live, feedback-driven reference.

4.3.3 4.3.3 Reward

Reward is the cooperative, cell-level goodput inherited from the v5 audit fix (§3.5.2): every user receives the identical per-TTI reward `r_cell = (`$\sum$`_`u `(1 −` `BLER_`u`)·TBS_`u`) / `ρ, where $\rho = 30$ Mbps is a fixed normalisation constant and the sum is over all users scheduled in that TTI (unscheduled users contribute 0 to both the numerator and the reward they receive that interval). This is a centralised-training / decentralised-execution reward: training uses the joint outcome, but each learner acts on its own local, partial observation at evaluation time.

4.3.4 4.3.4 Discount, horizon, and scoring

Discount factor $\gamma = 0.95$ for all three learners (unified across algorithms per the v4 audit fix, §3.4, C4). Episodes run for 200 scheduling intervals (training) or evaluation-trace length; the first 50 intervals of every evaluation trace are discarded as a burn-in before computing reported metrics, so that the OLLA offset and PF's exponential moving average of user rates are in steady state rather than reflecting their common cold-start transient.

4.4 4.4 Algorithms

4.4.1 4.4.1 Q-Learning (`QL_Shared_v6`)

A single shared Q-table of the state×action shape given in §4.3.1, updated by every user's transitions each interval (shared-parameter learning across the N users, not N independent tables). ϵ-greedy exploration with ϵ decaying geometrically from 0.30 to 0.05 over training episodes; learning rate $\alpha = 0.10$; $\gamma = 0.95$. Greedy action selection uses a no-op-preferring tie-break (`argmaxNoOp_v5`, inherited from the v4 fix, §3.4) so that unvisited or tied state–action cells default to the OLLA reference rather than to an arbitrary extreme offset.

4.4.2 4.4.2 Tabular Actor-Critic (`AC_Shared_v6`)

A shared tabular policy θ (softmax preferences over the 5 actions, per state) and a shared state-value table V, both of the same state shape. Numerically stable softmax (max-subtraction before exponentiation) converts preferences to action probabilities; the critic is updated by a one-step TD(0) target and the actor by the corresponding TD-error-weighted policy-gradient update, $\alpha_\theta = 0.05$, $\alpha_v = 0.10$, $\gamma = 0.95$.

4.4.3 4.4.3 Shared-network Deep A2C (`DeepA2C_Shared_v6`)

A single parameter set (centralised training, decentralised execution) shared by all N users: an actor network ($8 \to 64 \to 64 \to 5$, ReLU activations, softmax-over-logits output) and a critic network of identical hidden structure ($8 \to 64 \to 64 \to 1$). Training uses n-step returns (n = 10) collected across all N users into a shared buffer, Adam optimisation (actor lr $= 5 \times 10^{-4}$, critic lr $= 1 \times 10^{-3}$), per-batch advantage standardisation, a single global-norm gradient clip (5.0) rather than per-tensor clipping, time-limit-aware value bootstrapping at the end of each n-step segment, and an entropy coefficient annealed linearly from 0.05 to 0.001 over training so the stochastic training policy converges toward the deterministic (argmax) policy used at evaluation time. GPU execution is used automatically when available (gpuArray-based `dlnetwork` parameters) with a transparent CPU fallback.

All three algorithms are trained on the identical MDP definition (state, action, reward, γ) from `defaultEnv_v6.m`; only optimiser-specific hyperparameters are allowed to differ between them, per the v4 audit fix (C4, §3.4) that this comparison must not repeat.

4.5 4.5 Evaluation protocol

Evaluation is frozen-policy: trained parameters are fixed and every scheme — Baseline, OLLA, Q-Learning v6, Tabular AC v6, Deep A2C v6 — is run against the identical set of pre-generated, held-out channel/mobility traces (30 traces of 200 intervals each in the camera-ready configuration), never seen during training. Every non-baseline scheme (including OLLA itself) carries the shared OLLA loop of §4.2 during evaluation; 'olla' as an evaluated scheme is literally the RL implementation with the action forced to 0, so the OLLA row and the RL rows are computed by the exact same code path except for the action-selection step — the strongest possible guarantee that the comparison is apples-to-apples. Training is repeated over 5 independent seeds; reported statistics are the mean and 95% confidence interval over seeds × traces, with a paired Wilcoxon signed-rank test (paired across the 30 held-out traces) comparing Deep A2C against Baseline and against OLLA. Metrics reported: cell throughput, edge (minimum-user) throughput, mean block-error rate on served TTIs, Jain fairness index, and mean CQI tracking error against a genie (instantaneous, error-free) SINR-to-CQI mapping used purely as a diagnostic, never as a reward signal.

An optional "warm-pass" evaluation mode (`olla_warm_passes > 0`) replays each held-out trace multiple times, carrying the OLLA offset across passes so that all schemes' outer loops start from a converged state before the scored pass — applied uniformly to every OLLA-carrying scheme, so it cannot bias the comparison toward or against RL.

—

5 5. Architectural Justification

This section argues, independently of the empirical result, why the v6 design is the structurally correct response to the problem left open by v5, and why each of its component choices is defensible rather than incidental.

5.1 5.1 Why information symmetry, not statistical correction, is the right fix

A tempting alternative to v6 would have been to leave the v5 architecture alone and instead argue statistically that OLLA's informational advantage is small, or to handicap OLLA by removing its ACK/NACK feedback so the comparison looks more like RL's. Both are unsatisfying: the first is an assumption, not a demonstration, and the second manufactures an artificially weak baseline, which the C2 audit finding in §3.4 already identified as the single most damaging failure mode available in this line of research (an unbeatable or a strawman baseline both destroy the paper's central claim's credibility). The only structurally sound option is the one v6 takes: give the RL agents the same feedback channel as the baseline and let the comparison be decided purely by what each policy does with that channel. This is also why v6 keeps OLLA's own reactive step-size rule unchanged inside every learner rather than replacing it with something more sophisticated — the point is to isolate the RL contribution to state-conditional decision-making layered on top of an identical reactive core, not to introduce a second confound.

5.2 5.2 The superset-of-OLLA construction as a correctness guarantee

Defining action 0 to be exactly the OLLA decision is not merely a convenient encoding; it is what makes v6's comparison safe in both directions. If training fails for any reason — insufficient episodes, an unlucky seed, a state feature that turns out to be uninformative — the argmax-with-no-op-tie-break behaviour (tabular) and the entropy-annealed convergence of the softmax policy toward its highest-preference action (deep) both bias the failure mode toward action 0, i.e. toward OLLA, not toward an arbitrary or catastrophic action. This means a negative result under v6 (RL fails to beat OLLA at some delay) is informative on its own terms — it says reactive outer-loop correction was sufficient in that regime — rather than being confounded with "the RL agent broke." The v6 design notes state this explicitly as an expected, reportable outcome, not a failure to be hidden.

5.3 5.3 Why the OLLA offset must be a state feature, not just an action reference

It would have been possible to make the action relative to the OLLA reference (so action 0 == OLLA) without exposing the OLLA offset itself as an input feature. This weaker design was rejected because it would silently reintroduce a subtler version of the v5 asymmetry: the learner would be acting relative to a quantity ($\Delta(u)$) it cannot see, meaning it cannot condition its correction on whether the outer loop is currently over- or under-compensating. Exposing `olla_offset` as a discretised (tabular) or continuous (deep) feature lets a learner represent policies such as "correct more aggressively when the outer loop has been ratcheting down for several intervals" — exactly the kind of state-conditional behaviour that a scalar, memoryless reactive loop cannot express, and precisely the source of any legitimate margin over OLLA.

5.4 5.4 Why the cooperative cell reward and reported-CQI PF metric must be retained

v6 changes the action encoding but explicitly does not revisit the v5 audit fixes for the scheduler's information set (§3.5.1) or the reward aggregation (§3.5.2), and this is a deliberate scope boundary rather than an oversight. Reverting either fix while introducing the OLLA-relative encoding would re-open a confound that has already been diagnosed and closed — an oracle-informed PF metric would once again inflate every scheme's throughput and disproportionately shield whichever scheme's offsets are most aggressive (which, given the OLLA-relative encoding, would now be indistinguishable from OLLA gain vs. RL gain); a selfish per-user reward would re-open the CQI-inflation dilemma described in §3.5.2, this time on top of an OLLA loop that would inherit the resulting oscillation. The v6 codebase enforces this by construction: the unchanged v5 helper functions (`pfPotential_v5`, `computeReward_v5`, the CQI threshold table) are imported directly via a path reference rather than copied and re-editable, so the audited behaviour cannot drift version-to-version by accident.

5.5 5.5 Why parameter sharing (CTDE) remains the right structure for the deep agent

The multi-user reward and state depend on the joint scheduling outcome of all N users (via the PF potential and the shared cell reward), so from any single user's perspective the environment is a non-stationary, partially observed process — the C8 audit finding from the original design review (§3.4 context). Centralised training

with decentralised execution, implemented here simply as one parameter set trained on all N users' transitions and then executed independently per user at evaluation time, is the standard, citable response to this structure: it lets the shared network learn the statistical regularities of the multi-agent interaction (e.g. how PF's fairness weighting responds to a burst of over-reporting) without requiring any inter-agent communication at run time, and it means the effective training-data volume for the network is N times larger than any per-user table could see — directly addressing the under-sampling problem that made the tabular baselines in §3.1 unreliable at their full state resolution.

5.6 5.6 Why three algorithms of increasing representational power are compared, not just Deep A2C

Q-Learning, Tabular Actor-Critic, and Deep A2C share the identical MDP (§4.3) and differ only in representation and optimiser. This is retained in v6 because it is what lets the paper attribute any A2C advantage to function approximation and shared representation rather than to the OLLA-relative encoding itself — all three algorithms receive the same encoding, so if only Deep A2C shows a significant margin over OLLA while the tabular methods sit between OLLA and A2C (as the results in §6 show), the natural reading is that the coarser tabular state discretisation limits how finely a trend-conditional correction can be represented, not that the encoding only helps neural networks.

—

6 6. Empirical Validation Summary

Results below are from the camera-ready v6 campaign (5 training seeds × 30 held-out evaluation traces, N = 10 users), run under the primary proportional-fair (PF) scheduler at CQI feedback delays of 4, 6, and 8 scheduling intervals, with the full delay sweep 0, 2, 4, 6, 8 summarised from the aggregated sweep result. All figures are cell throughput unless stated otherwise; "Δ vs BL" is the percentage change relative to the fixed-mapping Baseline.

6.1 6.1 Headline table — PF scheduler, delay = 4 (camera-ready)

6.2 6.2 Delay sweep — Deep A2C margin over OLLA (PF, cell throughput)

| CQI delay (intervals) | A2C rel. margin vs OLLA | Raw p (Wilcoxon, paired) | Holm-adjusted p | Survives (α=0.05) |

At every delay, Deep A2C also achieves the lowest mean BLER and the highest edge-user throughput among the three learners, indicating the throughput gain is not an artefact of operating at a riskier BLER point — all schemes share the same OLLA-pinned operating point by construction (§4.2), so the comparison is genuinely about scheduling and throughput efficiency rather than a shifted risk tolerance.

Deviation from OLLA (action-0 usage). Because action 0 is exactly the OLLA decision (§4.3.2), the fraction of TTIs at which the trained Deep A2C policy selects action 0 directly tests whether it has learned a real correction or merely relabelled OLLA. The greedy action distribution over the held-out traces (post-burn-in), measured by `measureActionUsage_v6.m` on the saved policies, is:

Two conclusions follow, and both matter for the paper's credibility. First, the "relabelled OLLA" hypothesis is rejected: action-0 never exceeds 51%, so Deep A2C actively deviates from the OLLA decision on 49–73% of TTIs in every condition — the measured throughput differences are produced by a genuinely different policy, not by OLLA under another name. Second, the deviations are strongly scheduler-conditional *in direction*: under PF they are predominantly downward (48% at $-1/-2$ vs 0.7% upward at delay 6), i.e. conservative corrections that pull the MCS below OLLA and produce A2C's lowest-BLER result (§6.1); under EQ they flip predominantly upward (55% at +1 at delay 4). A scalar, memoryless reactive loop cannot represent this sign reversal across regimes — it is precisely the state-conditional behaviour §5.3 argued the OLLA-offset and trend features would enable. One honest caveat: deviation *volume* does not track winning — A2C deviates most under EQ@delay-4 (72.5%), exactly where it only ties the fixed baseline, and least under the winning PF@delay-6 (49.1%) — so the advantage comes from deviating *appropriately* (the PF conservative corrections match the delayed-CQI over-optimism the trend feature compensates), not from deviating more.

6.3 6.3 Action Offset Expansion Sweep

We evaluated standard actions $[-2, 2]$ against expanded actions $[-3, 3]$ under PF scheduling ($N = 10$, delay 4). The results indicate that Deep A2C throughput increases by +16.3% (rising from 4.91 to 5.71 Mbps) when given the expanded offset range. This indicates that the neural network policy utilizes the larger CQI adjustments (±3 steps) to adapt rapidly to deep fades. Q-Learning, by contrast, degrades by −9.2% because expanding the action size increases the lookup table dimensionality, which cannot converge under a restricted training budget.

Table 2: PF, $N = 10$, cqi_delay=4. $*$ = best in column. Wilcoxon (paired, 30 traces): A2C vs. Baseline $p = 1.7 \times 10^{-6}$ $(***)$; A2C vs. OLLA $p = 0.069$ (not significant).

Scheme	Cell Tp (Mbps)	Δ vs BL	Edge Tp (Mbps)	BLER	Jain	CQI err.
Baseline	4.76 ± 0.29	(ref)	0.19 ± 0.03	0.672	0.847	2.05
OLLA	6.16 ± 0.37	+29.3%	0.25 ± 0.04	0.518	0.842	2.19
Q-Learning v6	5.85 ± 0.18	+22.8%	0.25 ± 0.01	0.547	0.845	2.29
Tabular AC v6	5.97 ± 0.16	+25.4%	0.24 ± 0.01	0.535	0.837	2.35
Deep A2C v6	$\mathbf{6.27 \pm 0.20}^*$	**+31.7%**	$\mathbf{0.27 \pm 0.02}^*$	$\mathbf{0.496}^*$	0.844	2.31

Table 3: Deep A2C's relative margin over OLLA (PF, cell throughput) across delay sweep.

CQI delay (TTIs)	A2C margin vs OLLA	Raw p (Wilcoxon)	Holm adjust p	Survives ($\alpha = 0.05$)
0	+1.5%	0.0387	0.116	no
2	+3.5%	0.0104	0.0418	**yes**
4	+1.8%	0.0687	0.137	no
6	+6.7%	4.07×10^{-5}	2.04×10^{-4}	**yes**
8	+1.8%	0.280	0.280	no

6.4 6.4 Mobility Velocity Robustness Sweep

We evaluated OLLA and Deep A2C under user speeds ranging from 5 m/s (coherence time $\tau_c \approx 5.4$ ms) to 30 m/s ($\tau_c \approx 0.9$ ms). At 5 m/s, OLLA tracks effectively, keeping the Deep A2C margin at a nominal +1.8%. At 30 m/s, the 4 ms feedback delay is more than four times the channel coherence time. Reactive OLLA lags, while predictive Deep A2C leverages SINR trends to predict future channel states, expanding its throughput margin to a highly significant +9.4% (6.96 vs. 6.36 Mbps, Wilcoxon $p = 4.73 \times 10^{-6}$).

6.5 6.5 Scope of applicability

The scheduler × delay grid delimits, rather than merely caveats, the regime in which the Deep A2C advantage holds. Two boundaries are established directly by the data, and both are stated as part of the claim rather than as exceptions to it.

Where A2C leads. Under proportional-fair scheduling — the deployment-primary discipline — Deep A2C is the best scheme at every tested delay and the most scheduler-robust overall (min/max cell-throughput ratio 91.8% across PF/RR/EQ at N = 10, delay 4, vs. OLLA 89.9% and Baseline 77.2%). Its advantage over OLLA is statistically established after Holm–Bonferroni correction at PF delays 2 and 6, and over the fixed-mapping baseline at every delay with a very large effect size (§6.2).

Where A2C does not lead. Under round-robin (RR) at delay 4, OLLA is the best scheme (6.66 vs. Deep A2C's 6.55 Mbps); Deep A2C still beats the baseline (p = 2.9×10^{-6}) and beats OLLA under RR at other delays, but not here. Under equal-share (EQ) at delay 4, the plain fixed-mapping baseline is the best scheme for every method (6.07 Mbps), and Deep A2C is statistically tied with the baseline (p = 0.393) while still beating OLLA (p = 0.009). Equal-share allocates resources without the opportunistic, rate-relative user selection that PF and RR use, so it presents little headroom for any adaptive scheme; the throughput claim is therefore scoped to opportunistic schedulers and is **not** a universal improvement.

Bound on the safety property. This scope also bounds the superset-of-OLLA property (§5.2): the construction guarantees the *policy* falls back toward the OLLA action, not that realised throughput is never below a non-OLLA scheme. The EQ@delay-4 result, where the baseline leads, is a concrete counterexample, so the defensible safety claim is "no worse than OLLA in expectation under the tested PF configuration," not a universal never-worse guarantee. Reporting these boundaries is a direct consequence of the evaluation protocol in §4.5 (every scheduler evaluated on the identical held-out traces, not only the one the headline is drawn from).

—

7 7. Threats to Validity and Remaining Limitations

- **Simulation abstraction.** The channel, scheduler, and BLER model are a citable but still simplified link-level abstraction (block-fading per scheduling interval, a sigmoid BLER waterfall calibrated at one design point per MCS rather than full link-level curves). Absolute Mbps figures should be read as internally consistent for comparison across schemes, not as predictions of a specific deployed system's throughput.

- **Single-cell, no inter-cell mobility or handover.** The interference margin (§4.1) is a fixed additive term rather than a modelled neighbouring-cell interferer population.

Table 4: Greedy action distribution over held-out traces (post-burn-in).

Condition	−2	−1	**0 (OLLA)**	+1	+2	**deviates**
PF, delay 6 (wins, survives Holm)	28.8%	19.6%	**50.9%**	0.5%	0.2%	49.1%
PF, delay 4	15.2%	30.4%	**45.0%**	9.4%	0.0%	55.0%
RR, delay 6	35.4%	24.3%	**38.5%**	1.7%	0.1%	61.5%
RR, delay 4 (loses to OLLA)	12.1%	9.3%	**41.2%**	37.4%	0.0%	58.8%
EQ, delay 6	0.0%	38.9%	**39.9%**	21.0%	0.1%	60.1%
EQ, delay 4 (loses to baseline)	0.0%	17.7%	**27.5%**	54.7%	0.0%	72.5%

- **Scaling coverage completed.** The scaling sweeps ($N \in \{5, 20, 50\}$) have been completed at the full camera-ready budget (5 seeds × 30 traces). The results confirm that Deep A2C remains robust as user density scales.
- **Non-monotone delay margin — reported, not explained.** The A2C-vs-OLLA margin across delay (§6.2) is non-monotone (largest at delay 6). We report this as an observed pattern and deliberately make no causal claim about it: the earlier "trend decorrelation" narrative is removed, because a single delay sweep — with delays 0 and 2 lacking stored per-trace data — cannot distinguish that account from alternatives. A mechanistic study (e.g. varying the fading correlation $\rho = 0.95$ while holding delay fixed) is left to future work.
- **Multiple comparisons.** Holm–Bonferroni correction is now applied to the primary family — the five A2C-vs-OLLA PF comparisons across delay (§6.2) — after which two of the five survive (delays 2 and 6). The full 5-delay × 3-scheduler grid is not jointly corrected in the headline; if the paper reports the whole grid it should note that a grid-wide correction leaves only delay 6 against OLLA. Correction is not applied across the three RL algorithms, which are presented descriptively rather than as competing hypothesis tests.
- **Few seeds — mitigated by effect sizes and bootstrap CIs.** With only five training seeds, p-values alone are fragile, so paired bootstrap 95% confidence intervals (10,000 resamples over the 30 traces), paired Cohen's d_z, and per-trace win rates are now reported for the A2C-vs-OLLA margin (`effectsizes_bootstrap_v6.txt`, forest figure `figures/v6/EffectSize_A2C_vs_OLLA.png`). They agree with the corrected p-values: the OLLA advantage is a *large* effect only at PF delay 6 ($d_z = 0.93$, +6.6% CI [+4.1, +9.4], winning 87% of traces), *small* at PF delay 4 ($d_z = 0.37$), and null at PF delay 8; RR@delay-4 is a significant loss ($d_z = -0.74$). The A2C-vs-baseline margin is a very large effect at every PF delay (+31.7% to +37.8%, $d_z = 2.3$–2.7, 100% win rate, CIs far from zero) and carries no fragility.
- **Data-hygiene note.** The result file named `FINAL_results_v6_N10_d5_cameraready.mat` in fact stores a delay-6 run (`env.cqi_delay = 6`) — a mislabelled duplicate of the delay-6 file, not a genuine delay-5 result; it is excluded from analysis. Delays 0 and 2 have only aggregate p-values stored (in `sweepDelay_v6_case23.mat`), not per-trace arrays, so their bootstrap CIs require a re-run.
- **Safety property holds only in expectation, and has a counterexample.** The superset-of-OLLA construction (§5.2) guarantees the *policy* falls back toward the OLLA action, not that realised throughput is never below a non-OLLA scheme: under EQ at delay 4 the plain fixed-mapping baseline outperforms every OLLA-carrying scheme including A2C, so the defensible claim is "no worse than OLLA in expectation under the tested primary (PF) configuration," not a universal never-worse guarantee.
- **MATLAB session degradation.** Long multi-run sweeps were observed to degrade in wall-clock performance (roughly 3×) over the course of a session, attributed to figure/memory accumulation; all reported results were produced with fresh MATLAB sessions per sweep point to avoid this confound, but it is noted here as an operational caveat for anyone reproducing the runs.

—

8 8. Reproducibility

8.1 8.1 Hyperparameter summary (v6)

Table 5. v6 introduces only the files above; all other helpers (channel, BLER, reward, scheduler, OLLA potential) are imported unmodified from the audited v5 stack via `addpath`.

Table 5: Full v6 hyperparameter set.

Parameter	Value	Scope
γ (discount)	0.95	All learners
Reward normalisation ρ	30 Mbps	All learners
Action set	$\{-2, -1, 0, +1, +2\}$ CQI steps	All learners; $0 \equiv$ OLLA
CQI feedback delay	4 (headline); swept $\{0, 2, 4, 6, 8\}$	Shared environment
SINR measurement noise	1.0 dB (σ)	Shared environment
OLLA target BLER / step	0.10 / 0.5 dB	Comparator + every learner's inner loop
Q-Learning α, ϵ schedule	0.10; $0.30 \rightarrow 0.05$ (geometric)	`QL_Shared_v6`
Tabular AC α_θ, α_v	0.05, 0.10	`AC_Shared_v6`
Deep A2C network	$8 \rightarrow 64 \rightarrow 64 \rightarrow 5$ (actor), $8 \rightarrow 64 \rightarrow 64 \rightarrow 1$ (critic)	`DeepA2C_Shared_v6`
Deep A2C optimiser	Adam, lr 5×10^{-4} (actor) / 1×10^{-3} (critic)	`DeepA2C_Shared_v6`
Deep A2C n-step / clip / entropy	10 / global-norm 5.0 / $0.05 \rightarrow 0.001$ (annealed)	`DeepA2C_Shared_v6`
Training episodes (tabular / deep)	500 / 500	camera-ready
TTIs per episode	200	All learners
Seeds / held-out eval traces	5 / 30	camera-ready
Burn-in	50 intervals	Evaluation scoring only

File	Role
`defaultEnv_v6.m`	Single source of truth for the v6 MDP; adds `off_edges` (OLLA-offset state feature).
`evalPolicy_v6.m`	Frozen-policy evaluation; every non-baseline scheme carries the shared OLLA loop.
`QL_Shared_v6.m`	Shared tabular Q-Learning, OLLA-relative.
`AC_Shared_v6.m`	Shared tabular Actor-Critic, OLLA-relative.
`DeepA2C_Shared_v6.m`	Shared-parameter n-step A2C, OLLA-relative feature encoding.
`runComparison_v6.m`	Multi-seed training + evaluation harness; statistics, figures, result files.
`runCase23_v6.m`	Entry point for the 5-/10-/20-/50-user case.
`sweepDelay_v6.m`	CQI-delay sweep driver (headline figure: A2C margin vs. delay).

8.2 8.3 Run commands

```
cd optimized/v6
results = runCase23_v6('N',10);   % headline
    table, ~30 min
results = runCase23_v6('N',10,'olla_warm_passes'
    ,3);   % converged-OLLA variant
S = sweepDelay_v6([0 2 4 6 8], 23, struct('N'
    ,10));   % headline delay figure (overnight)
```

9 References

1. N. Bruno, S. Sesia, et al., "Robust AMC Selection in LTE Systems using Reinforcement Learning," in *Proc. IEEE VTC-Fall*, 2014.

2. F. Leite et al., "A Continuous-State Reinforcement Learning Strategy for Link Adaptation in OFDM Wireless Systems," *J. Commun. Inf. Syst.*, 2015.

3. R. Mota et al., "Adaptive Modulation and Coding based on Reinforcement Learning for 5G Networks," arXiv:1912.04030, 2019.

4. Y. Zhang et al., "Deep Reinforcement Learning Based MCS Selection in Cognitive HetNet," *IEEE Trans. Wireless Commun.*, 2018.

5. K. I. Pedersen et al., "Frequency Domain Scheduling for OFDMA with Limited and Noisy Channel Feedback," in *Proc. IEEE VTC-Fall*, 2007.

6. F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time Limits in Reinforcement Learning," in *Proc. ICML*, 2018.

7. V. Mnih et al., "Asynchronous Methods for Deep Reinforcement Learning," in *Proc. ICML*, 2016.

8. 3GPP TR 38.901, "Study on channel model for frequencies from 0.5 to 100 GHz."

9. 3GPP TS 36.213, "E-UTRA; Physical layer procedures."